

ЛЕКЦИЯ 10

Планирование задач

Инженер описывает словарь действий — последовательность строит алгоритм

две части по 45

ение робототехническими комплексами»

План занятия

1 Третий уровень автоматизации логики

2 Модель STRIPS и предположение о замкнутости мира

3 Язык PDDL: домен и задача

4 Сложность и почему всё держится на эвристиках

5 Релаксация с отбрасыванием удалений

6 Семейство эвристик: $h_{\max}$, h_{add} , h_{FF} , ориентиры

7 Иерархические сети задач HTN

8 Временные сети STN и STNU

9 Планирование в контуре: перепланирование и стабильность

10 Разделение ролей: планировщик и супервизор

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Словарь действий вместо последовательности

Модель STRIPS, язык PDDL и источник всех эвристик планирования.

Действие STRIPS и контракт навыка



Единый источник истины: одно описание для планировщика, исполнителя и монитора.

10.1 Три уровня автоматизации логики

Глава	Что задаёт инженер	Что вычисляется
3	модель оборудования + запреты	супервизор
8	предположения о среде + цели на LTL	реактивный контроллер
10	действия с pre/post + цель	последовательность действий

Все три раза человек отвечает за постановку, машина — за перебор

10.2 Модель STRIPS

- $\Pi = (F, A, s_0, g)$: атомы, действия, начальное состояние, цель
- Действие $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$
- Применимость: $\text{pre}(a) \subseteq s$; результат $\gamma(s, a) = (s \setminus \text{del}(a)) \cup \text{add}(a)$
- **Предположение о замкнутости мира**: всё, что не указано, ложно
- Состояние — множество истинных атомов

Факт

Предположение о замкнутости мира — источник типовой ошибки при моделировании РТК: «предмет не обнаружен» и «предмета нет» кодируются одинаково, хотя различие принципиально для восприятия.

10.3 Словарь действий для примера A

- ``pick(o, t)``: pre — `at_robot(t)`, `on(o,t)`, `gripper_empty`, `detected(o)`
- add — `holding(o)`; del — `on(o,t)`, `gripper_empty`
- ``move(from, to)``: pre — `at_robot(from)`, `connected(from,to)`
- ``place(o, b)``: pre — `at_robot(b)`, `holding(o)`, `type_matches(o,b)`
- ``observe(o, l)``: даёт `detected(o)` — действие сбора информации

Ключевая идея

Это в точности контракт навыка из главы 1: pre и post действия для планировщика — те же самые pre и post, что реализованы узлами-условиями дерева поведения. Единое описание исключает рассогласование.

- **Домен** — типы, предикаты, действия: что вообще возможно
- **Задача** — объекты, начальное состояние, цель: что требуется сейчас
- Разделение позволяет менять обстановку, не трогая описание действий
- Числовые функции: стоимости, времена, метрика минимизации
- Уровни: STRIPS → ADL → PDDL 2.1 → PDDL+ → PDDL 3

Ключевая идея

Начинайте с STRIPS и добавляйте выразительность только при необходимости. Каждый уровень резко сужает круг доступных планировщиков и увеличивает время поиска.

Существование плана — PSPACE-полно; плана длины $\leq k$ — NP-полно

- PSPACE-трудность сводит принятие линейно-ограниченной машины Тьюринга
- План, если существует, имеет длину не более $2^{|F|}$
- Практическая ценность целиком лежит в качестве эвристик
- Отсюда — весь дальнейший материал первой части

Планирование принципиально дорого, и это не лечится вычислительной мощностью

$$h^+(s) \leq h^*(s)$$

- В Π^+ полагаем $\text{del}(a) = \emptyset$: достигнутый атом остаётся истинным навсегда
- Всякий план исходной задачи работает и в релаксации — множество решений шире
- Значит, минимум по большему множеству не больше: эвристика **допустима**
- Но вычисление h^+ само NP-трудно

Основной источник всех современных эвристик планирования

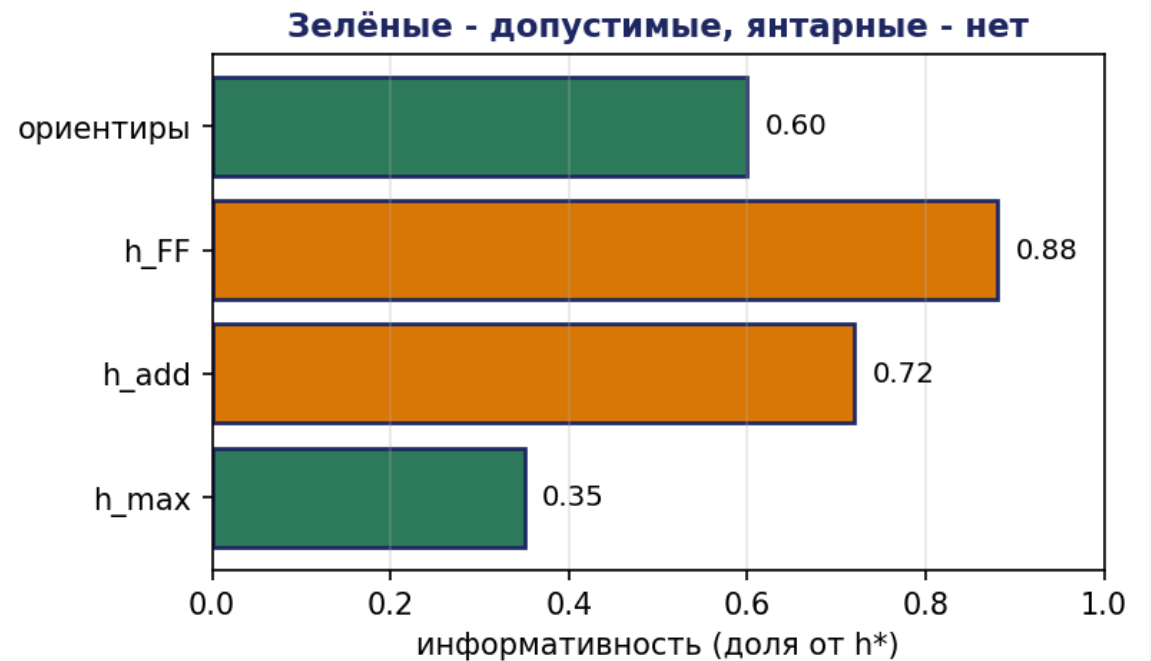
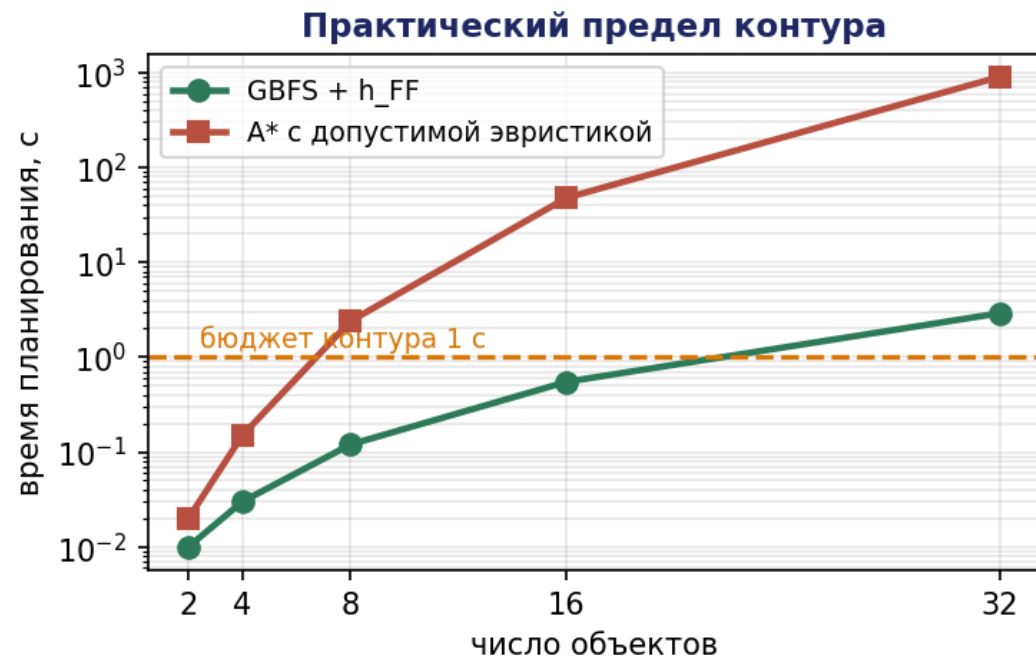
10.7 Семейство эвристик

Эвристика	Идея	Свойства
$h_{\max}$	стоимость самой дорогой подцели	допустима, но сильно занижает
h_{add}	сумма стоимостей подцелей	недопустима, но информативнее
h_{FF}	длина релаксированного плана	недопустима, эмпирически лучшая
Ориентиры	факты, обязательные в любом решении	допустима при разделимости
Абстракции	проекция на подмножество переменных	допустима, дороже

h_{FF} дополнительно порождает «полезные действия», резко сужающие ветвление

10.7 Предел планирования и эвристики

Вся практическая ценность планирования лежит в эвристиках



Измерьте свой предел заранее - не на демонстрации перед заказчиком.



Выбираем режим

Робот должен перепланировать миссию за время не более 1 секунды. Какой режим поиска выбрать?

A

A* с допустимой эвристикой — гарантия оптимальности

B

Жадный поиск с h_{FF} — быстро, план неоптимален

C

Полный перебор — надёжнее всего

D

Никакой: за секунду планирование невозможно



Выбираем режим

Робот должен перепланировать миссию за время не более 1 секунды. Какой режим поиска выбрать?

A

A* с допустимой эвристикой — гарантия оптимальности

B

Жадный поиск с h_{FF} — быстро, план неоптимален

C

Полный перебор — надёжнее всего

D

Никакой: за секунду планирование невозможно

Ответ. Второй. Для доменов масштаба РТК GBFS с h_{FF} даёт план за доли секунды; A* может занимать минуты. Оптимизация выполняется на уровне диспетчеризации, а не в контуре.

П Е Р Е Р Ы В

Конец первой части

Мы разобрали классическое планирование. После перерыва — иерархии, время и включение планировщика в контур.

ЧАСТЬ 2

Иерархии, время и контур

HTN, временные сети ограничений и место планировщика в работающей системе.

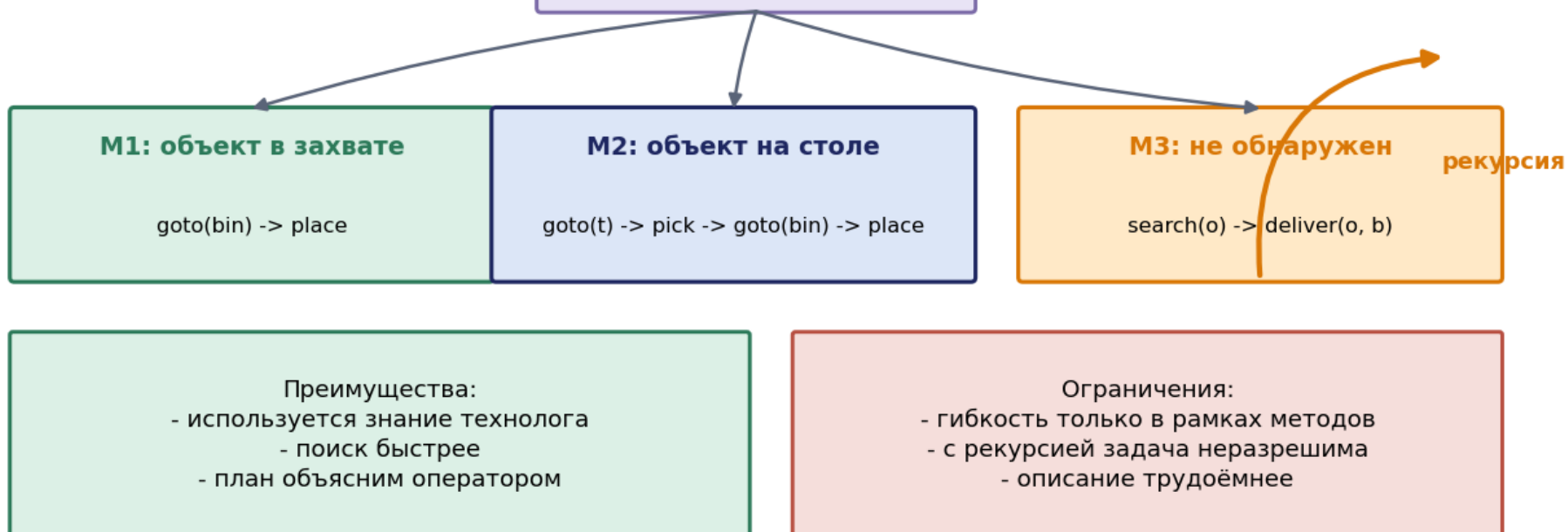
10.8 Иерархические сети задач (HTN)

- Задачи делятся на примитивные (действия) и составные
- **Метод** указывает, как составная задача декомпозируется при условии
- План строится рекурсивным применением методов до примитивных задач
- Инженер задаёт не только действия, но и структуру решения
- Пример: `deliver(o,b)` → три метода в зависимости от того, где объект

Пример

Метод «объект не обнаружен»: `pre` — `not(detected o)`; подзадачи — `[search(o), deliver(o,b)]`. Рекурсивная декомпозиция: сначала найти, затем поставить задачу заново.

HTN: инженер задаёт структуру решения, а не только действия
deliver(o, b)



Знание технолога записывается явно, поиск ускоряется, план объясним оператором.

10.9

Классическое планирование против HTN

Критерий	PDDL	HTN
Использование знаний предметной области	слабое	сильное
Скорость поиска	зависит от эвристики	обычно выше
Гибкость в новых ситуациях	высокая	ограничена набором методов
Трудоёмкость описания	ниже	выше
Объяснимость плана	средняя	высокая: видна иерархия
Разрешимость	PSPACE	неразрешима с рекурсией

Пример Б (маршрут задан технологом) — HTN; пример А (произвольная расстановка) — классическое

10.10 Простые временные сети (STN)

- Переменные — моменты времени; ограничения $a \leq x_j - x_i \leq b$
- Граф расстояний: дуга $i \rightarrow j$ веса b и дуга $j \rightarrow i$ веса $-a$
- Совместность $\Leftrightarrow$ отсутствие цикла отрицательного веса
- Решение алгоритмом кратчайших путей за $O(n^3)$
- Даёт **множество** допустимых расписаний, а не одно

Ключевая идея

Множество допустимых расписаний позволяет исполнителю выбирать моменты в реальном времени, поглощая отклонения. Жёсткое расписание такой возможности не даёт.

10.11 Неопределённые длительности: STNU

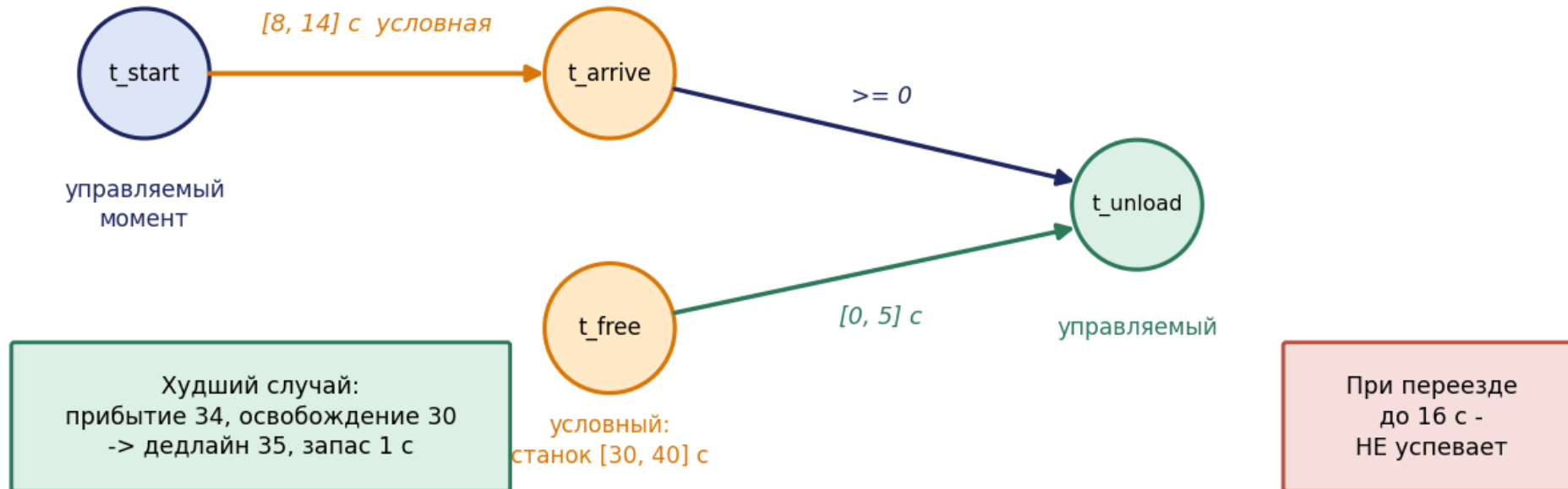
- Длительность переезда не выбирается — она «от 8 до 14 с»
- Такие связи называются **условными** (contingent)
- **Динамическая контролируемость**: стратегия, зависящая от уже наблюдаемых исходов
- Проверяется за полиномиальное время
- Отрицательный ответ означает: расписание слишком жёсткое

Ключевая идея

Это тот анализ, которого не хватает в типовом проекте: расписание, составленное по номинальным длительностям, оказывается неисполнимым при их реальном разбросе, и выясняется это на пусконаладке.

10.11 Динамическая контролируемость

STNU: успеет ли расписание при неточных длительностях



Запас 1 секунда. При переезде до 16 с сеть уже не контролируема.



Считаем запас

Станок освободится в $[30, 40]$ с, переезд занимает $[8, 14]$ с, выгрузка допустима не позже чем через 5 с после освобождения. Робот выезжает в момент 20. Успеет ли?

А

Да, с большим запасом

В

Да, но запас всего 1 секунда

С

Нет, не успеет

Д

Зависит от того, когда именно освободится станок



Считаем запас

Станок освободится в $[30, 40]$ с, переезд занимает $[8, 14]$ с, выгрузка допустима не позже чем через 5 с после освобождения. Робот выезжает в момент 20. Успеет ли?

A

Да, с большим запасом

B

Да, но запас всего 1 секунда

C

Нет, не успеет

D

Зависит от того, когда именно освободится станок

Ответ. Прибытие в $[28, 34]$. Худший случай: прибытие 34, освобождение 30, дедлайн 35 — успевает с запасом 1 с. При переезде до 16 с уже не успел бы.

10.12 Планирование в контуре



Здесь смыкаются главы 6 и 10: действие плана исполняется под деревом, а контракт навыка служит его описанием

10.13

Когда перепланировать

- Не по таймеру, а **по событиям**
- Нарушено предусловие текущего действия
- Действие завершилось отказом после исчерпания повторов
- Обнаружен новый объект, меняющий цель
- Изменилась доступность ресурсов — отказ манипулятора

Ключевая идея

Планировщик работает на порядок медленнее исполнителя — правило разделения масштабов времени из главы 1 применяется здесь буквально.

10.14 Стабильность плана как требование

- Полное перепланирование избыточно при локальном изменении
- Починка плана (plan repair) — поиск минимального изменения
- Скользящий горизонт: планировать только ближайшие шаги
- Инкрементальный поиск (D* Lite) для планов-путей
- Стабильность важнее оптимальности для доверия персонала

Факт

Непредсказуемо мечущийся робот вызывает недоверие персонала независимо от формальной оптимальности его решений. Стабильность плана — самостоятельное эксплуатационное требование.

10.15 Временной домен для примера Б

- Длительные действия: load (5 с), process (22 с), unload (5 с)
- Зона моделируется предикатом zone-free
- `at start (not (zone-free))` — захват в начале действия
- `at end (zone-free)` — освобождение в конце
- Это в точности конструкция, порождавшая сифон в главе 4

Ключевая идея

Планировщик строит только исполнимые планы и потому тупика в плане не будет. Но он **не гарантирует** отсутствия тупика при исполнении, где действия могут задержаться.

“

Планировщик отвечает за эффективность. За безопасность отвечает структурный механизм.

Роли не взаимозаменяемы и не дублируются

10.16 Планировщики

Планировщик	Особенности
Fast Downward	эталонная архитектура, множество эвристик, PDDL 2.2
FF	классический, h_FF, очень быстрый
LAMA	ориентиры, режим «якорь и улучшение»
pyperplan	учебный, Python, читаемый исходный код
OPTIC / POPF	временные, PDDL 2.1 с длительностями
PlanSys2	интеграция планирования и исполнения в ROS 2

На семинаре 6 — *pyperplan* для понимания устройства, *Fast Downward* для реальных задач

10.17 Масштаб задач

< 0,5 с

план для 6 предметов, GBFS +
h_FF

минуты

оптимальный план A* для той
же задачи

10–12

действий в типовом плане
примера A

экспонента

рост времени с числом
объектов

- Практический предел контура — 15–30 объектов
- Дальше требуется декомпозиция задачи или иерархия

10.18 Типовые ошибки при описании домена

- Забыт атом «обследовано» — путаница отсутствия информации и отсутствия объекта
- Слишком детальные предикаты — взрыв пространства состояний
- Геометрические условия закодированы булевыми атомами без обоснования
- Стоимости не заданы — планировщик выдаёт формально корректный, но абсурдный план
- Домен и дерево поведения описывают действие по-разному

Факт

Третья ошибка ведёт прямо в тему следующего занятия: булев атом `reachable(o, l)` скрывает геометрию, и планировщик выдаст план, который исполнитель не сможет выполнить.

10.19

Практическая рекомендация по объёму домена

- 5–10 типов действий покрывают миссию мобильного манипулятора
- 15–25 предикатов достаточно; больше — признак избыточной детализации
- Числовые стоимости обязательны — иначе оптимизируется длина, а не время
- Домен пишется один раз и живёт весь проект
- Задача генерируется автоматически из состояния мировой модели

“

Описание действия пишется один раз и используется трижды:
планировщиком, исполнителем и монитором.

Контракт навыка — единый источник истины

Выводы

- 1 Планирование позволяет описать словарь действий вместо последовательности
- 2 Существование плана PSPACE-полно; вся практическая ценность в эвристиках
- 3 Релаксация с отбрасыванием удалений даёт допустимую, но NP-трудную эвристику; применяются аппроксимации
- 4 HTN использует знание технолога: быстрее и объяснимее, но менее гибок
- 5 STN даёт множество допустимых расписаний; STNU проверяет исполнимость при неточных длительностях
- 6 Планировщик включается в контур по событиям, а не по таймеру; стабильность плана — требование
- 7 Планировщик отвечает за цель, супервизор — за невозможность недопустимого

Вопросы для самопроверки

1. Что означает предположение о замкнутости мира и какие ошибки оно порождает?
2. Сформулируйте задачу STRIPS; зачем в действии и add, и del?
3. Почему релаксация с отбрасыванием удалений даёт допустимую эвристику? Докажите
4. Чем h_{add} отличается от h_{max} и почему обычно эффективнее, будучи недопустимой?
5. Что такое полезные действия и как они ускоряют поиск?
6. Когда HTN предпочтительнее классического планирования?
7. Почему HTN с рекурсией неразрешим и как это ограничивают?
8. Что даёт STN по сравнению с фиксированным расписанием?
9. Объясните смысл динамической контролируемости и приведите неконтролируемый пример
10. Почему стабильность плана — эксплуатационное требование, а не удобство?